Kreiranje prve Android aplikacije

Cilj ove vježbe je upoznavanje studenata s osnovnim konceptima Android razvoja aplikacija. Po završetku vježbe studenti će znati šta su osnovni gradivni elementi Android aplikacije, kako se dizajnira korisnički interfejs, te kako se koristi Android Studio okruženje zajedno s emulatorom i debug alatima.

1. Uvod u Android

Android je operativni sistem baziran na modificiranoj verziji Linux kernela, dizajniran za pametne telefone, tablete i ostale mobilne uređaje. Pojavio se 2008. godine i do danas je objavljen veliki broj verzija - svaka verzija dobiva kodno ime abecednim redom (Cupcake, Donut, Eclair, Froyo, Gingerbread...). Danas su aktuelne verzije Android 12, 13, 14 i 15.

1.1 Programski jezici i alati

Android aplikacije su se tradicionalno pisale u Javi, međutim danas je Kotlin preferirani i preporučeni jezik za razvoj Android aplikacija. Google je 2019. proglasio Kotlin jezikom prvog izbora (Kotlin-first).

Jezik / Alat	Opis
Kotlin	Preporučeni jezik za Android razvoj - sažet, siguran i potpuno kompatibilan sa Javom
Java	Tradicionalni jezik za Android, i dalje podržan
Android SDK	Skup alata i biblioteka potrebnih za razvoj Android aplikacija
Android Studio	Zvanični IDE za Android razvoj, baziran na IntelliJ IDEA
Gradle	Automatizovani build sistem za kompajliranje, testiranje i pakovanje aplikacije
APK / AAB	Krajnji format paketa aplikacije - APK za direktnu instalaciju, AAB za Google Play

1.2 Gradle i struktura build fajlova

Gradle je build sistem koji Android Studio automatski konfiguriše. Najvažniji fajlovi su:

- **build.gradle (projekt nivo)** - globalne postavke projekta i lista repozitorija
- **build.gradle (app modul)** - postavke modula: compileSdkVersion, minSdkVersion, targetSdkVersion, dependencies
- **settings.gradle** - lista svih modula koji su dio projekta

> **Napomena:** *compileSdkVersion označava verziju SDK-a za kompajliranje. minSdkVersion je minimalna verzija Androida na uređaju. targetSdkVersion je verzija za koju je aplikacija optimizovana i testirana.*

2. Komponente Android aplikacije

Android aplikacija se sastoji od više tipova komponenti. Svaka komponenta ima jasno definisanu ulogu i vlastiti životni ciklus.

2.1 Activity (Aktivnost)

Activity predstavlja jedan ekran aplikacije s korisničkim interfejsom. Svaka aktivnost je nezavisna cjelina koja upravlja vlastitim layoutom, UI elementima i korisničkim interakcijama. Aplikacija može imati više aktivnosti - npr. u news aplikaciji jedna aktivnost prikazuje listu vijesti, druga detalje odabranog članka, treća postavke aplikacije.

Svaka aktivnost prolazi kroz definisani životni ciklus koji se sastoji od niza metoda koje Android sistem poziva u određenim momentima.

Razumijevanje životnog ciklusa ključno je za ispravno upravljanje resursima aplikacije.

Metoda	Kada se poziva	Tipična upotreba
onCreate()	Aktivnost se kreira prvi put	Inicijalizacija UI elemenata, postavljanje layouta (setContentView), kreiranje ViewModela
onStart()	Aktivnost postaje vidljiva	Registracija listenera, pokretanje animacija
onResume()	Aktivnost je u fokusu i interaktivna	Nastavak pauziranih operacija, ažuriranje podataka
onPause()	Aktivnost gubi fokus (druga je ispred)	Čuvanje nezavršenih unosa korisnika
onStop()	Aktivnost više nije vidljiva	Otpuštanje skupe obrade (npr. zaustavljanje videa)
onDestroy()	Aktivnost se uništava	Konačno čišćenje resursa, otpuštanje referenci

Najvažnija metoda je onCreate() - poziva se jednom pri pokretanju aktivnosti i tu se obavlja sva inicijalizacija. Prima parametar savedInstanceState koji sadrži prethodno sačuvano stanje (npr. pri rotaciji ekrana).

```
class MainActivity : AppCompatActivity() {

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Postavljanje XML layouta kao UI aktivnosti
        setContentView(R.layout.activity_main)
        // Inicijalizacija UI elemenata i logike
    }

    override fun onResume() {
        super.onResume()
        // Poziva se svaki put kada aktivnost postane aktivna
```

```
        // npr. povratak s druge aktivnosti
    }

    override fun onPause() {
        super.onPause()
        // Čuvanje privremenih podataka prije odlaska
    }
}
```

Korisno: *Svaka aplikacija mora imati barem jednu aktivnost označenu kao LAUNCHER u AndroidManifest.xml - to je aktivnost koja se pokreće pokretanjem aplikacije.*

2.2 View i ViewGroup

View je osnovna gradivna jedinica korisničkog interfejsa u Androidu. Svaki element koji korisnik vidi na ekranu - dugme, tekst, slika, polje za unos je objekat klase View ili neke od njenih podklasa.

View zauzima pravougaoni prostor na ekranu i odgovoran je za vlastito crtanje i obradu korisničkih događaja (klik, dugi klik, dodir...).

ViewGroup je posebna vrsta View-a koja služi kao kontejner za druge View objekte. ViewGroup ne prikazuje vlastiti sadržaj, već organizuje i raspoređuje child View elemente prema pravilima koja su specifična za svaki tip layout-a.

Neki od najčešće korištenih View elemenata:

Klasa	Opis
TextView	Prikaz teksta koji korisnik ne može mijenjati
EditText	Polje za unos i uređivanje teksta (podržava hint, inputType...)
Button	Dugme koje reaguje na klik
ImageView	Prikaz slike iz drawable resursa ili URL-a
CheckBox	Prekidač da/ne, može se odabrati više istovremeno
RadioButton	Jedna opcija iz grupe (u kombinaciji s RadioGroup)
Switch	Moderniji toggle prekidač (alternativa CheckBox-u)
ProgressBar	Prikaz napretka operacije (određen ili neodređen)

Svaki View ima skup XML atributa kojima se definiše izgled i ponašanje. Najvažniji su layout_width i layout_height koji mogu imati vrijednosti match_parent (zauzmi cijelu širinu/visinu roditelja), wrap_content (zauzmi samo onoliko prostora koliko treba sadržaju) ili konkretnu dimenziju u dp jedinicama.

```
<!-- Primjer: tekstualno polje za pretragu vijesti -->
<EditText
    android:id="@+id/etSearch"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:hint="@string/search_hint"
    android:inputType="text"
    android:textSize="16sp"
    android:padding="12dp" />
```

Napomena: *Dimenzije u Androidu se uvijek navode u dp (density-independent pixels) za layout elemente i sp (scale-independent pixels) za veličine teksta. Ove jedinice automatski se skaliraju u zavisnosti od gustine ekrana uređaja, što osigurava konzistentan izgled na svim uređajima.*

Slika 1. Prikaz Korisničkog interfejsa

2.3 Intent

Intent je mehanizam komunikacije između komponenti Android sistema. Koristi se za pokretanje aktivnosti, servisa, slanje poruka ili otvaranje URL-a. Postoje eksplicitni intenti (specificirana ciljana klasa) i implicitni intenti (samo akcija, sistem pronalazi ko može odgovoriti).

2.4 Fragment

Fragment je modularni dio aktivnosti koji ima vlastiti životni ciklus i layout. Jedna aktivnost može prikazivati više fragmenata istovremeno.

Za razliku od aktivnosti fragment ne može postojati samostalno jer uvijek mora biti prikačen na aktivnost.

2.5 Service (Servis)

Servis je komponenta koja radi u pozadini bez korisničkog interfejsa. Koristi se za dugotrajne operacije poput preuzimanja podataka s interneta, reprodukcije muzike ili sinhronizacije podataka s udaljenim serverom.

Primjer: U aplikaciji za vijesti servis se koristi za periodično dohvatanje novih vijesti u pozadini i prikazivanje notifikacije kada stignu breaking news.

2.6 Content Provider

Content Provider upravlja dijeljenjem podataka između različitih aplikacija. Podatke može čuvati u bazi (SQLite/Room), fajlovima ili na mreži. Na ovaj način npr. aplikacija za kontakte dijeli podatke s aplikacijom za pozivanje.

2.7 Broadcast Receiver

Broadcast Receiver sluša sistemske ili aplikacijske broadcast poruke. Primjeri: baterija je prazna, internet veza je prekinuta, uređaj se upalio. Receiver može pokrenuti drugu komponentu ili prikazati notifikaciju.

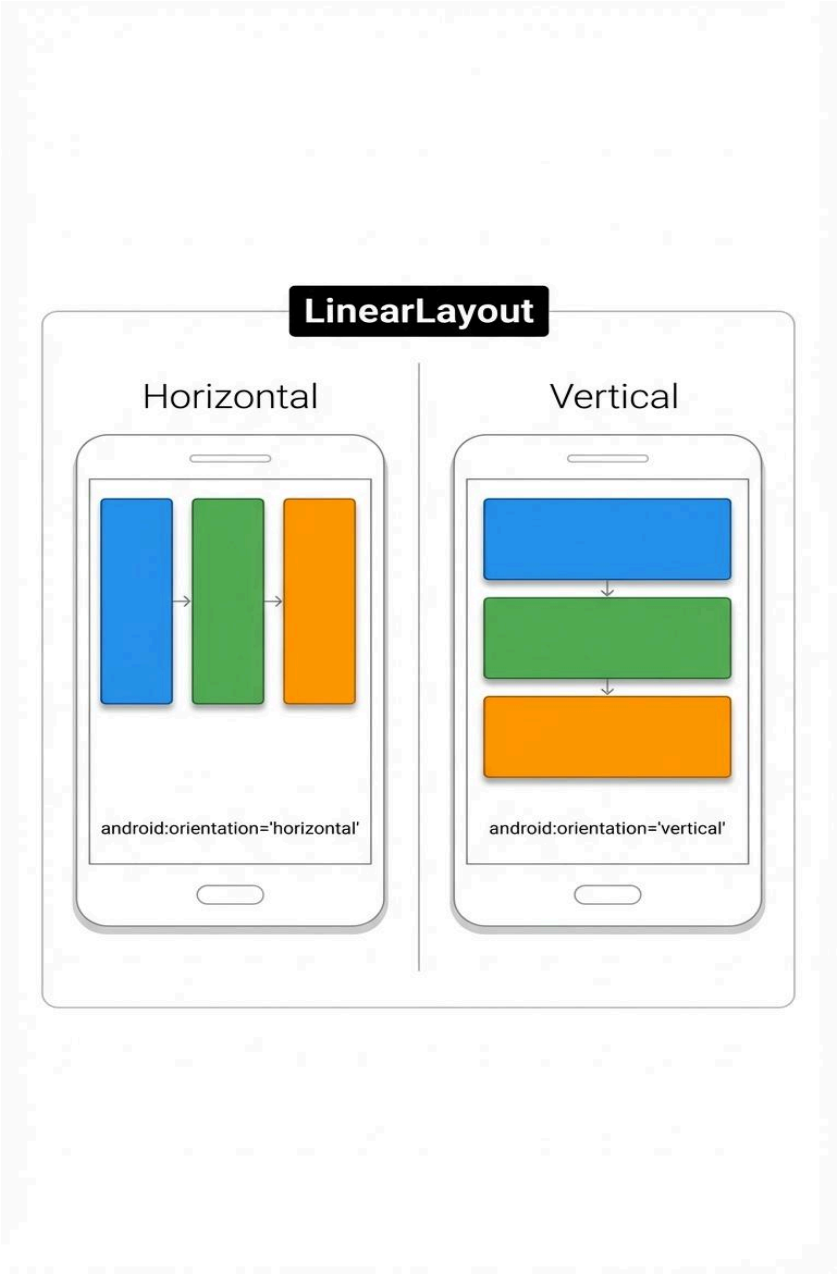
2.8 AndroidManifest.xml

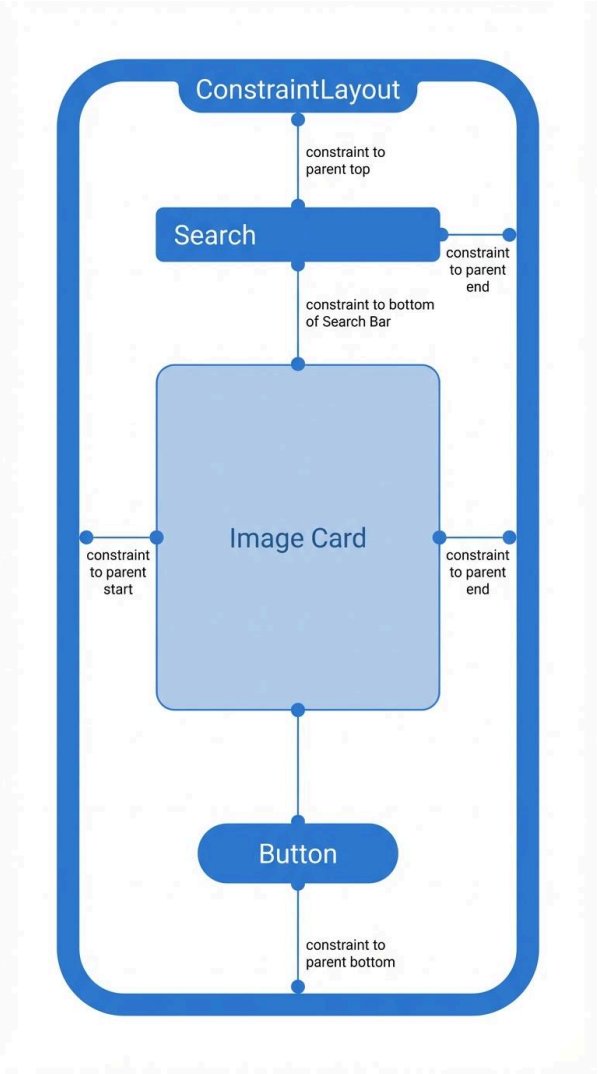
Centralni konfiguracioni fajl svake Android aplikacije. Sadrži deklaracije svih aktivnosti, servisa, receivera i content providera, permisije koje aplikacija zahtijeva (kamera, internet, lokacija...), minimalne i ciljane verzije Androida, te ikonu i naziv aplikacije.

3. Android korisnički interfejs

3.1 Najvažniji layout-i

Layout	Opis
LinearLayout	Raspoređuje elemente u jednom smjeru - horizontalno ili vertikalno. Jednostavan ali ograničen za kompleksne ekrane.
RelativeLayout	Elementi se pozicioniraju relativno jedan prema drugom ili prema roditeljskom elementu.
ConstraintLayout	Najmoćniji layout. Elementi se vezuju ograničenjima za roditelja ili susjedne elemente. Preporučen za kompleksne ekrane.
FrameLayout	Jednostavan kontejner - elementi se slažu jedan na drugi. Koristi se za fragmente i overlay sadržaj.





3.2 Ulazne kontrole

- **Button** - standardno dugme za izvršavanje akcije
- **EditText** - polje za unos teksta (tekst, broj, lozinka...)
- **TextView** - prikaz teksta (samo za čitanje)
- **ImageView** - prikaz slike
- **CheckBox** - prekidač da/ne, može se odabrati više opcija
- **RadioButton / RadioGroup** — bira se samo jedna opcija iz grupe
- **Spinner** - padajuća lista za odabir jedne vrijednosti
- **Switch** - moderni toggle prekidač (alternativa CheckBox-u)

3.3 Event listener-i

Interakcija korisnika detektuje se putem event listener-a. Listener je interfejs s callback metodom koja se poziva kad se dogodi odgovarajuća akcija. Najčešće korišten je OnClickListener:

```
button.setOnClickListener {
    // akcija koja se izvršava na klik
    textView.text = "Kliknuto!"
}
```

Pored klika, postoje i drugi listener-i: OnLongClickListener, OnFocusChangeListener, OnTouchListener i TextWatcher (za EditText).

3.3 Prikaz liste podataka

Kada je potrebno prikazati veći broj stavki podataka u scrollabilnoj listi, Android nudi dvije komponente: stariji **ListView** i moderniji **RecyclerView**.

ListView

ListView je starija komponenta koja automatski prikazuje niz podataka u vertikalnoj scrollabilnoj listi. Svaki element liste se kreira pozivanjem adaptera koji podatke pretvara u odgovarajući View. Mana ListView-a je što kreira novi View objekat za svaki element čak i kada taj element nije vidljiv na ekranu, što kod velikih listi uzrokuje usporavanje aplikacije i povećanu potrošnju memorije. U modernom Android razvoju ListView se gotovo više ne koristi - zamijenjen je RecyclerView-om.

RecyclerView

RecyclerView je napredna i preporučena zamjena za ListView. Ključna razlika je u tome što RecyclerView **reciklira** View objekte - kada element izađe s ekrana pri scrollanju, njegov View se ne uništava nego se ponovo iskoristi za novi element koji ulazi na ekran. Ovaj mehanizam drastično smanjuje potrošnju memorije i čini scroll glatkim čak i za liste s hiljadama elemenata. RecyclerView zahtijeva nešto više koda za postavljanje (Adapter + ViewHolder + LayoutManager), ali ta investicija se isplati u performansama i fleksibilnosti.

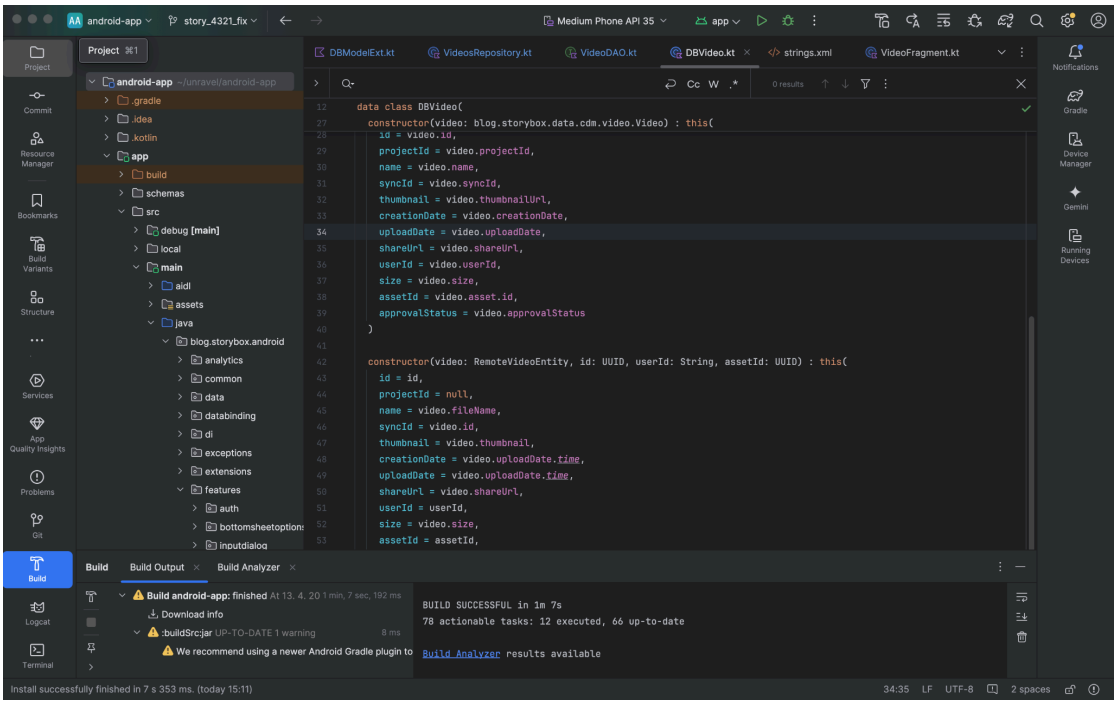
4. Android Studio okruženje

Android Studio je zvanični IDE za Android, baziran na IntelliJ IDEA. Sadrži sve potrebno za razvoj - editor koda, emulator, debugger, profiler i alate za dizajn layouta.

4.1 Ključni dijelovi sučelja

- **Toolbar** - pokretanje, zaustavljanje i debug aplikacije, odabir uređaja
- **Project panel** - pregled strukture projekta i svih fajlova
- **Editor** - uređivač koda i XML layouta (Code / Split / Design prikaz)

- **Logcat** - prikaz log poruka iz aplikacije u realnom vremenu
- **Debug panel** - pregled varijabli, call stack-a i breakpoint-a tokom debug sesije



4.2 Struktura Android projekta

- **manifests/** — AndroidManifest.xml
- **kotlin+java/** — Kotlin i Java izvorni kod, organizovan po paketima
- **res/layout/** — XML fajlovi korisničkog interfejsa
- **res/values/** — strings.xml, colors.xml, themes.xml
- **res/drawable/** — slike i vektorske ikone
- **res/mipmap/** — ikone aplikacije različitih veličina

5. Debug i logiranje

Debug je proces pronalaženja i otklanjanja grešaka u aplikaciji. Android Studio nudi napredne debug alate.

5.1 Breakpoint-i

Breakpoint je oznaka na određenoj liniji koda gdje će se izvršavanje pauzirati. Postavlja se klikom na marginu pored linije koda. Debug panel prikazuje trenutno stanje svih varijabli.

5.2 Logcat

Logcat prikazuje log poruke iz aplikacije i sistema u realnom vremenu. Logovi se pišu pozivanjem metoda klase Log:

Metoda	Namjena
Log.e(tag, msg)	Error - kritična greška, aplikacija možda neće moći nastaviti rad
Log.w(tag, msg)	Warning - upozorenje, nešto nije kako bi trebalo biti
Log.i(tag, msg)	Info - informativna poruka o toku izvršavanja
Log.d(tag, msg)	Debug - poruka za debugiranje, ukloniti prije produkcije
Log.v(tag, msg)	Verbose - detaljne poruke, samo za intenzivno debugiranje

```
// Primjer logiranja
val TAG = "NewsApp"
Log.d(TAG, "Učitano vijesti: ${articles.size}")
Log.e(TAG, "Greška pri dohvatanju podataka", exception)
```

Korisno: *U Logcat-u je moguće filtrirati poruke po tagu, nivou logiranja ili paketu aplikacije. Ovo olakšava pronalaženje relevantnih poruka u velikom broju sistemskih logova.*